Projeto de exemplo: mini CRM em Rails e em Vetra

O mesmo mini CRM multiempresa implementado em Rails 8.1 (código real e idiomático) e em Vetra (sintaxe proposta, que ainda não existe), para comparar esforço, conceitos e o que cada um garante.

Escopo

Um mini CRM de leads para empresas com várias revendas, exposto como API JSON.

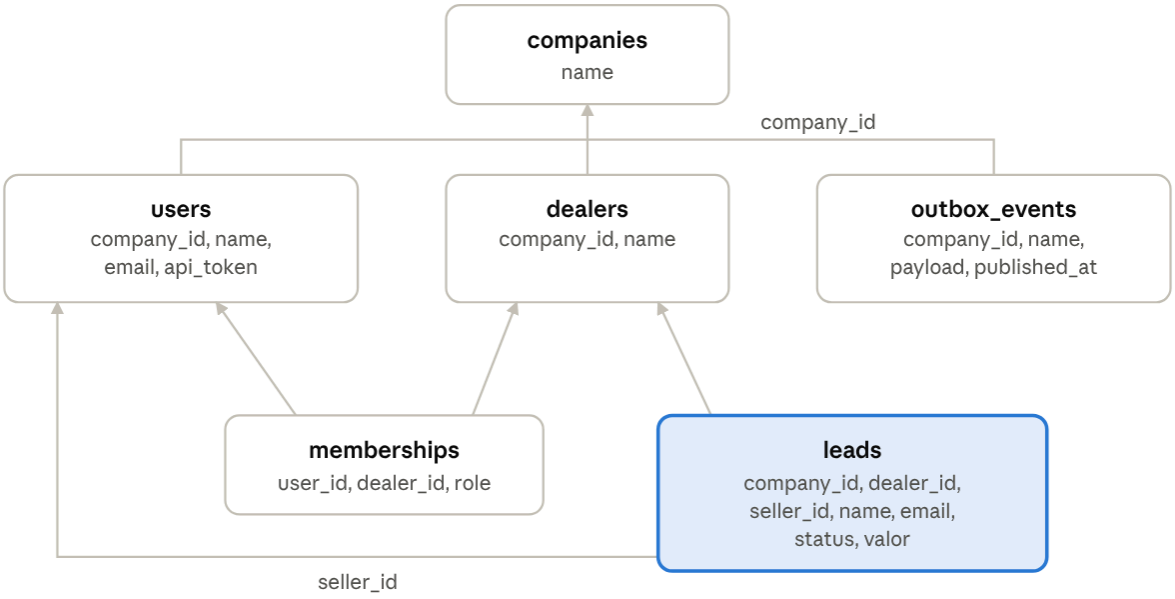
- **Multiempresa:** cada empresa só vê seus dados; usuários atuam em uma ou mais revendas.
- **Papéis por revenda:** vendedor lê, cria e edita leads; gerente também exclui.
- **Lead:** nome, e-mail (dado pessoal), status, valor estimado em reais, revenda e vendedor.
- **Regra de negócio:** "ganhar lead" só vale para leads em aberto e gera o evento `crm.lead.won`.
- **Efeito assíncrono:** ao ganhar um lead, o vendedor recebe um e-mail, entregue via outbox.
- **Dinheiro:** API recebe e devolve valores canônicos (`{ amount: "1234.56", currency: "BRL" }`); a formatação fica com o frontend.

Fora do exemplo nos dois lados: cadastro de usuários e revendas, template do e-mail e frontend.

Arquitetura do projeto

O modelo de dados é o mesmo nos dois frameworks; a diferença está em quem escreve cada parte: no Rails, 17 arquivos organizados por tipo técnico; no Vetra, 5 arquivos em um único domínio, com o resto gerado.

Leads, revendas e usuários pertencem sempre a uma empresa
Modelo de dados do Exemplo 1 · setas apontam para a tabela referenciada



modelo de dados do mini CRM · 6 tabelas

No Rails, nada no banco impede um lead de apontar para a revenda de outra empresa; isso depende da validação no model. No Vetra, as FKs compostas (`company_id` , `id`) e o RLS garantem isso no próprio banco.

As árvores abaixo mostram o projeto completo, como no explorador do editor. Marcações: `# escrito` é código deste exemplo; `# editado` é arquivo gerado e alterado à mão; itens sem marcação são gerados e não alterados. Pastas sem relação com o exemplo aparecem fechadas, com uma nota do que contêm.

Estrutura em Rails 8.1

```
mini_crm/
├── .github/                          # CI e Dependabot
├── .kamal/                           # hooks e segredos do deploy
├── app/
│   ├── controllers/
│   │   ├── concerns/
│   │   ├── application_controller.rb    # escrito
│   │   └── leads_controller.rb          # escrito
│   ├── jobs/
│   │   ├── application_job.rb
│   │   ├── notify_seller_job.rb         # escrito
│   │   └── outbox_relay_job.rb          # escrito
│   ├── mailers/
│   │   └── application_mailer.rb
│   ├── models/
```

```
│   │   ├── concerns/
│   │   ├── application_record.rb
│   │   ├── company.rb                    # escrito
│   │   ├── current.rb                    # escrito
│   │   ├── dealer.rb                     # escrito
│   │   ├── lead.rb                       # escrito
│   │   ├── membership.rb                 # escrito
│   │   ├── outbox_event.rb               # escrito
│   │   └── user.rb                       # escrito
│   ├── policies/
│   │   ├── application_policy.rb         # gerado pelo pundit:install
│   │   └── lead_policy.rb                # escrito
│   ├── services/
│   │   └── leads/
│   │       └── win.rb                    # escrito
│   └── views/                            # só layouts de e-mail (modo API)
├── bin/                                  # rails, rake, setup, dev, jobs,
kamal, ci
├── config/
│   ├── environments/
│   │   ├── development.rb
│   │   ├── production.rb
│   │   └── test.rb
│   ├── initializers/
│   │   ├── cors.rb
│   │   ├── filter_parameter_logging.rb
│   │   ├── inflections.rb
│   │   └── money.rb                      # escrito
│   ├── locales/
│   ├── application.rb
│   ├── boot.rb
│   ├── cable.yml
│   ├── cache.yml
│   ├── ci.rb
│   ├── credentials.yml.enc
│   ├── database.yml
│   ├── deploy.yml
│   ├── environment.rb
│   ├── master.key
│   ├── puma.rb
│   ├── queue.yml
│   ├── recurring.yml                     # editado
│   ├── routes.rb                         # editado
│   └── storage.yml
├── db/
│   ├── migrate/
```

```
|   |   └── 20261029000000_create_crm.rb # escrito
|   ├── cable_schema.rb
|   ├── cache_schema.rb
|   ├── queue_schema.rb
|   ├── schema.rb
|   └── seeds.rb
├── lib/
|   └── tasks/
├── log/
├── public/                             # robots.txt
├── script/
├── storage/
├── test/                               # controllers, models, fixtures,
test_helper.rb
├── tmp/
├── vendor/
├── .dockerignore
├── .gitattributes
├── .gitignore
├── .rubocop.yml
├── .ruby-version
├── config.ru
├── Dockerfile
├── Gemfile                             # alterado pelo bundle add
├── Gemfile.lock
├── Rakefile
└── README.md
```

Estrutura em Vetra

```
mini_crm/
├── .vetra/
|   ├── cache/
|   └── graph.json                    # IR versionada do grafo
├── app/
|   └── domains/
|       └── crm/
|           ├── dealer.rb             # escrito · entity
|           ├── lead.rb               # escrito · entity
|           ├── notify_seller.rb      # escrito · handler
|           ├── roles.rb              # escrito · roles
|           └── win_lead.rb           # escrito · command
├── bin/
|   ├── setup
|   └── vetra
├── config/
```

```
|   ├── environments/
|   |   ├── development.rb
|   |   ├── production.rb
|   |   └── test.rb
|   ├── locales/
|   |   └── pt-BR.yml
|   ├── app.rb                          # tenant, moeda, idioma, auth
|   ├── database.rb                     # papéis do banco: aplicação e
migração
|   ├── secrets.enc                     # segredos cifrados, inclusive a chave
do contexto assinado
|   └── server.rb                       # Falcon
├── db/
|   ├── plans/                          # planos de migração expand/contract
|   ├── policies/                       # políticas RLS geradas, em SQL
revisável
|   ├── schema.sql                      # retrato do banco
|   └── seeds.rb
├── lib/
├── log/
├── public/
├── sdk/
|   └── typescript/                     # cliente tipado para Vue e React
├── storage/
├── test/
|   ├── domains/
|   |   └── crm/                        # testes escritos pela equipe
|   ├── generated/                      # testes de autorização e isolamento
gerados do grafo
|   └── test_helper.rb
├── tmp/
├── .gitignore
├── .ruby-version                       # 4.0.9
├── config.ru
├── Dockerfile
├── Gemfile
├── Gemfile.lock
└── README.md
```

Empresas, usuários e vínculos com revenda não aparecem na árvore do Vetra porque são entidades geradas pela opção `--auth=token`.

Rails 8.1 — criação do projeto, gems e migração

Tudo neste lado é escrito à mão, exceto o que os geradores criam (`ApplicationPolicy`, configuração do Solid Queue).

```
rails new mini_crm --api --database=postgresql
cd mini_crm
bundle add pundit money-rails
bin/rails generate pundit:install
bin/rails generate migration CreateCrm
bin/rails db:create db:migrate
```

```
# config/initializers/money.rb
MoneyRails.configure do |config|
  config.default_currency = :brl
  config.rounding_mode = BigDecimal::ROUND_HALF_UP
end
```

```
# db/migrate/20261029000000_create_crm.rb
class CreateCrm < ActiveRecord::Migration[8.1]
  def change
    create_table :companies do |t|
      t.string :name, null: false
      t.timestamps
    end

    create_table :dealers do |t|
      t.references :company, null: false, foreign_key: true
      t.string :name, null: false
      t.timestamps
    end

    create_table :users do |t|
      t.references :company, null: false, foreign_key: true
      t.string :name, null: false
      t.string :email, null: false, index: { unique: true }
      t.string :api_token, null: false, index: { unique: true }
      t.timestamps
    end

    create_table :memberships do |t|
      t.references :user, null: false, foreign_key: true
      t.references :dealer, null: false, foreign_key: true
      t.string :role, null: false
      t.timestamps
```

```
      t.index %i[user_id dealer_id], unique: true
    end

    create_table :leads do |t|
      t.references :company, null: false, foreign_key: true
      t.references :dealer, null: false, foreign_key: true
      t.references :seller, foreign_key: { to_table: :users }
      t.string :name, null: false
      t.string :email
      t.string :status, null: false, default: "new"
      t.monetize :valor, amount: { null: true, default: nil },
                         currency: { null: true, default: nil }
      t.timestamps
      t.index %i[company_id status]
    end

    create_table :outbox_events do |t|
      t.references :company, null: false, foreign_key: true
      t.string :name, null: false
      t.jsonb :payload, null: false, default: {}
      t.datetime :published_at
      t.timestamps
      t.index :published_at, where: "published_at IS NULL"
    end
  end
end
```

Note que a FK de `dealer` não garante que a revenda seja da mesma empresa do lead; isso fica para uma validação no model.

Rails 8.1 — models, Current e autorização

O isolamento entre empresas depende de dois lugares estarem certos: a validação no model e o escopo na política.

```
# app/models/company.rb
class Company < ApplicationRecord
  has_many :dealers
  has_many :users
  has_many :leads
end
```

```
# app/models/dealer.rb
class Dealer < ApplicationRecord
  belongs_to :company
  has_many :memberships
end
```

```
# app/models/user.rb
class User < ApplicationRecord
  belongs_to :company
  has_many :memberships
  has_secure_token :api_token

  def dealer_ids_for(*roles)
    memberships.where(role: roles.map(&:to_s)).pluck(:dealer_id)
  end
end
```

```
# app/models/membership.rb
class Membership < ApplicationRecord
  ROLES = %w[vendedor gerente].freeze

  belongs_to :user
  belongs_to :dealer

  validates :role, inclusion: { in: ROLES }
end
```

```
# app/models/lead.rb
class Lead < ApplicationRecord
  belongs_to :company
  belongs_to :dealer
  belongs_to :seller, class_name: "User", optional: true

  enum :status, %w[new contacted won lost].index_by(&:itself),
       default: "new", prefix: true

  monetize :valor_cents, allow_nil: true

  validates :name, presence: true, length: { in: 2..160 }
  validates :email, format: { with: URI::MailTo::EMAIL_REGEXP }, allow_blank:
true
  validate :same_company

  def closed? = status_won? || status_lost?
```

```
  private

  def same_company
    if dealer && dealer.company_id != company_id
      errors.add(:dealer, "não pertence à empresa")
    end
    if seller && seller.company_id != company_id
      errors.add(:seller, "não pertence à empresa")
    end
  end
end
```

```
# app/models/outbox_event.rb
class OutboxEvent < ApplicationRecord
  belongs_to :company
end
```

```
# app/models/current.rb
class Current < ActiveSupport::CurrentAttributes
  attribute :user
end
```

```
# app/policies/lead_policy.rb
class LeadPolicy < ApplicationPolicy
  def show?    = in_dealer?(:vendedor, :gerente)
  def create?  = in_dealer?(:vendedor, :gerente)
  def update?  = in_dealer?(:vendedor, :gerente)
  def win?     = in_dealer?(:vendedor, :gerente)
  def destroy? = in_dealer?(:gerente)

  class Scope < ApplicationPolicy::Scope
    def resolve
      scope.where(company_id: user.company_id,
                  dealer_id: user.dealer_ids_for(:vendedor, :gerente))
    end
  end

  private

  def in_dealer?(*roles)
    record.company_id == user.company_id &&
      user.dealer_ids_for(*roles).include?(record.dealer_id)
  end
end
```

Rails 8.1 — controllers, serviço, outbox, jobs e rotas

O ponto mais sutil está no `update`: é preciso reautorizar depois de atribuir os parâmetros, ou um vendedor move o lead para uma revenda onde não tem acesso.

```
# app/controllers/application_controller.rb
class ApplicationController < ActionController::API
  include ActionController::HttpAuthentication::Token::ControllerMethods
  include Pundit::Authorization

  before_action :authenticate!
  after_action :verify_authorized, except: :index
  after_action :verify_policy_scoped, only: :index

  rescue_from Pundit::NotAuthorizedError do
    render json: { error: "forbidden" }, status: :forbidden
  end

  private

  def authenticate!
    Current.user = authenticate_with_http_token do |token, _options|
      User.find_by(api_token: token)
    end
    head :unauthorized unless Current.user
  end

  def pundit_user = Current.user
end
```

```
# app/controllers/leads_controller.rb
class LeadsController < ApplicationController
  before_action :set_lead, only: %i[show update destroy win]

  def index
    leads = policy_scope(Lead).order(created_at: :desc).limit(50)
    render json: leads.map { |lead| lead_json(lead) }
  end

  def show
    render json: lead_json(@lead)
  end

  def create
    lead = Current.user.company.leads.new(lead_params)
    authorize lead
```

```
    if lead.save
      render json: lead_json(lead), status: :created
    else
      render json: { errors: lead.errors }, status: :unprocessable_content
    end
  end

  def update
    @lead.assign_attributes(lead_params)
    authorize @lead # reautoriza: a revenda pode ter mudado
    if @lead.save
      render json: lead_json(@lead)
    else
      render json: { errors: @lead.errors }, status: :unprocessable_content
    end
  end

  def destroy
    @lead.destroy!
    head :no_content
  end

  def win
    if Leads::Win.call(lead: @lead, actor: Current.user)
      render json: lead_json(@lead)
    else
      render json: { errors: @lead.errors }, status: :unprocessable_content
    end
  end

  private

  def set_lead
    @lead = policy_scope(Lead).find(params[:id])
    authorize @lead
  end

  def lead_params
    attrs = params.expect(lead: [:name, :email, :dealer_id, :seller_id,
                                 { valor: [:amount, :currency] }])
    if (valor = attrs.delete(:valor))
      attrs[:valor] = Money.from_amount(BigDecimal(valor[:amount]),
                                        valor[:currency] || "BRL")
    end
    attrs
  end
```

```
  def lead_json(lead)
    {
      id: lead.id,
      name: lead.name,
      email: lead.email,
      status: lead.status,
      dealer_id: lead.dealer_id,
      seller_id: lead.seller_id,
      valor: lead.valor && {
        amount: ActiveSupport::NumberHelper.number_to_rounded(
          lead.valor.amount, precision: lead.valor.currency.exponent
        ),
        currency: lead.valor.currency.iso_code
      }
    }
  end
end
```

```
# app/services/leads/win.rb
module Leads
  class Win
    def self.call(lead:, actor:)
      Lead.transaction do
        lead.lock!
        if lead.closed?
          lead.errors.add(:status, "lead já encerrado")
          raise ActiveRecord::Rollback
        end

        lead.update!(status: "won")
        OutboxEvent.create!(
          company: lead.company,
          name: "crm.lead.won",
          payload: { lead_id: lead.id, actor_id: actor.id }
        )
      end
      lead.errors.empty?
    end
  end
end
```

```
# app/jobs/outbox_relay_job.rb
class OutboxRelayJob < ApplicationJob
  def perform
    OutboxEvent.transaction do
```

```
      OutboxEvent.where(published_at: nil).order(:id).limit(100)
                 .lock("FOR UPDATE SKIP LOCKED").each do |event|
        dispatch(event)
        event.update!(published_at: Time.current)
      end
    end
  end

  private

  def dispatch(event)
    case event.name
    when "crm.lead.won"
      NotifySellerJob.perform_later(event.payload["lead_id"])
    end
  end
end
```

```
# app/jobs/notify_seller_job.rb
class NotifySellerJob < ApplicationJob
  def perform(lead_id)
    lead = Lead.find(lead_id)
    LeadMailer.won(lead).deliver_now if lead.seller
  end
end
```

```
# config/recurring.yml (Solid Queue)
production:
  outbox_relay:
    class: OutboxRelayJob
    schedule: every 10 seconds
```

```
# config/routes.rb
Rails.application.routes.draw do
  resources :leads do
    post :win, on: :member
  end
end
```

Limites deste lado: sem RLS (o isolamento depende só da política e da validação), sem histórico de alterações, e o `NotifySellerJob` pode enviar e-mail duplicado se o relay reprocessar um evento — a idempotência fica a cargo da aplicação.

Vetra — criação do projeto e entidades (camada 1)

Sintaxe proposta: nada abaixo existe ainda. Na camada 1, o desenvolvedor só declara entidades; o CRUD inteiro é gerado.

```
vetra new mini_crm --tenant=company --currency=BRL --locale=pt-BR --
auth=token
cd mini_crm
vetra db:plan      # mostra o SQL que será aplicado, em etapas
expand/contract
vetra db:migrate
vetra dev
```

Arquivo gerado pelo `vetra new`, não escrito à mão:

```
# config/app.rb
Vetra.app :mini_crm do
  tenant   :company
  currency "BRL", rounding: :half_up
  locale   "pt-BR"
  auth     :token, memberships: :dealer
end
```

A opção `--auth=token` também gera empresas, usuários e vínculos usuário–revenda com papel.

```
# app/domains/crm/dealer.rb
entity :dealer do
  tenant :company
  string :name, required: true
end
```

```
# app/domains/crm/lead.rb
entity :lead do
  tenant :company
  belongs_to :dealer, same_tenant: true
  belongs_to :seller, entity: :user, optional: true, same_tenant: true

  string :name,   required: true, length: 2..160
  email  :email,  optional: true, personal: true
  enum   :status, %w[new contacted won lost], default: "new",
                  groups: { closed: %w[won lost] },
                  guarded: { "won" => :win_lead }   # "won" só pelo comando
  money  :valor,  optional: true

  crud scope: :dealer
  index :company_id, :status
end
```

O que essas duas declarações geram:

- Tabelas com FKs compostas (`company_id` , `id`), que garantem revenda e vendedor da mesma empresa no próprio banco.
- Políticas RLS com contexto assinado e negação por padrão.
- Rotas `GET/POST/PATCH/DELETE /leads` , contratos de entrada e respostas 422 padronizadas.
- Autorização reavaliada sobre o estado final no `update` — o problema da troca de revenda não existe.
- `valor` canônico na API (`{ amount, currency }`), BRL por padrão, arredondamento comercial.
- `email` mascarado em logs, traces e erros; histórico de alterações; OpenAPI e tipos TypeScript.

Campo de transição (ADR-021): `guarded:` faz o valor `won` ser aceito só pelo comando `win_lead` . O `update` do CRUD gerado recusa `status: "won"` , e o evento `crm.lead.won` não pode ser pulado. No Rails deste exemplo o problema não existia, porque `lead_params` não permite o `status` .

Vetra — papéis, comando e handler (camada 2)

A camada 2 aparece só quando há regra de negócio além do CRUD: aqui, ganhar um lead e notificar o vendedor.

```
# app/domains/crm/roles.rb
roles scope: :dealer do
  role :vendedor do
    can :read, :create, :update, on: :lead
  end

  role :gerente do
    can :manage, on: :lead
  end
end
```

```
# app/domains/crm/win_lead.rb
command :win_lead do
  input :lead_id, :id
  authorize :update, :lead, record: :lead_id
  uses :leads

  emits "crm.lead.won", with: %i[id seller_id]

  call do |input, leads|
    lead = leads.lock!(input.lead_id)
    fail! :status, "lead já encerrado" if lead.closed?
    leads.update!(lead, status: "won")
  end

  expose http: "POST /leads/:lead_id/win"
end
```

```
# app/domains/crm/notify_seller.rb
handler :notify_seller do
  on "crm.lead.won"
  uses :mailer
  idempotent_by :event_id

  call do |event, mailer|
    next unless event.seller_id

    mailer.deliver :lead_won, to: event.seller_id, lead_id: event.id
  end
end
```

Implícito pelo framework: transação no comando, gravação do evento na outbox, relay e agendamento, `fail!` convertido em resposta 422, deduplicação do handler pela chave do evento (sem chave de idempotência aceita pelo provedor de e-mail, uma queda entre o

envio e o registro ainda pode duplicar o e-mail), e o `mailer` como única saída externa do handler (Effect Boundary).

Comparação

O Rails pede cerca de 300 linhas em 17 arquivos; o Vetra, cerca de 55 linhas em 5 arquivos — com mais garantias. Mas o código Vetra é hipotético, e a diferença depende de geradores e convenções que são justamente a parte mais difícil de construir.

Medida	Rails 8.1	Vetra (proposta)
Arquivos escritos à mão	17	5
Linhas aproximadas	300	55
Conceitos para o CRUD multiempresa	~10 (migração, model, associação, validação, enum, monetize, Current, controller, parâmetros, política e escopo)	2 (entidade e papéis)
Conceitos para a regra e o e-mail	+5 (serviço, transação com lock, outbox, jobs, agendamento)	+2 (comando e handler)

Garantia	Rails 8.1	Vetra (proposta)
Isolamento entre empresas	Política + validação manual	FK composta + RLS + escopo gerado
Troca de revenda no update	Reautorização manual, fácil de esquecer	Automática
Dinheiro canônico na API	Conversão e serialização manuais	Tipo `money`
E-mail em logs	Filtrado nos parâmetros por padrão	Mascarado em logs, traces e erros
Histórico de alterações	Ausente	Gerado
Evento durável	Outbox, relay e agendamento manuais	`emits`

Garantia	Rails 8.1	Vetra (proposta)
Reenvio do e-mail	Sem deduplicação	Deduplicado por evento (`idempotent_by`); exatamente uma vez só com idempotência no provedor
OpenAPI e tipos TypeScript	Ausente	Gerados

Leitura honesta: um app Rails maduro reduziria a diferença com geradores de scaffold, uma gem de serialização e uma gem de outbox. A contagem de linhas é só observação; a métrica que decide continua sendo o tempo para implementar uma mudança de regra com testes e segurança preservados. Este projeto é um bom candidato para o módulo de referência das fases 1 e 2 do roadmap.